Primo 1: Carga del móvil (`while`)

Concepto: `while` con condición previa y bucle que puede no ejecutarse.

Enunciado: escribe un programa Java con la clase `Main`. Un móvil se carga por tandas y queremos seguir el progreso. Debes crear este método dentro de `Main`:

```
static void cargar(int faltante)
```

El parámetro `faltante` es el porcentaje de batería que falta por cargar. El método debe seguir estas reglas:

- Declara un contador `tanda` que empieza en 0.
- Usa un `while` que se repita mientras `faltante` sea mayor que 0.
- En cada vuelta suma 1 a `tanda`, resta 20 a `faltante` e imprime `Tanda N: falta X%`. `N` es el número de tanda y `X` es el valor de `faltante` ya actualizado. No hay espacio entre el número y el símbolo `%`.
- Al terminar el bucle, si `tanda` vale 0, imprime exactamente `Ya estaba cargado, qué milagro.`
- Si no vale 0, imprime `Móvil cargado en N tandas.`

El orden de impresión en `main` es:

1. Imprime `Miércoles:`.
2. Llama a `cargar(60)`.
3. Imprime `Jueves:`.
4. Llama a `cargar(0)`.

Salida prevista:

text

```
Miércoles:
Tanda 1: falta 40%
Tanda 2: falta 20%
Tanda 3: falta 0%
Móvil cargado en 3 tandas.
Jueves:
Ya estaba cargado, qué milagro.
```

Primo 2: Noche de karaoke (`do-while`)

Concepto: `do-while`, con el cuerpo ejecutado siempre al menos una vez.

Enunciado: escribe un programa Java con la clase `Main`. En un karaoke hay una lista de canciones pendientes y alguien se empeña en cantar. Debes crear este método dentro de `Main`:

```
static void cantar(int canciones)
```

El método debe usar obligatoriamente un bucle `do { ... } while (...);` con estas reglas:

- Dentro del cuerpo, si `canciones` es mayor que 0, resta 1 a `canciones` e imprime `Canción cantada, quedan N`. `N` es el valor ya actualizado.
- Si `canciones` no es mayor que 0, imprime exactamente `No había canciones, pero el micrófono ya estaba encendido.`
- La condición del `while` final es `canciones > 0`.

El orden de impresión en `main` es:

1. Imprime `Karaoke 1:`.
2. Llama a `cantar(4)`.
3. Imprime `Karaoke 2:`.
4. Llama a `cantar(0)`.

Salida prevista:

text

```
Karaoke 1:
Canción cantada, quedan 3
Canción cantada, quedan 2
Canción cantada, quedan 1
Canción cantada, quedan 0
Karaoke 2:
No había canciones, pero el micrófono ya estaba encendido.
```

PROMETEO
by thePower

Primo 3: Las llaves perdidas (`break`)

Concepto: `break` para salir de un bucle y valor centinela (`-1`).

Enunciado: escribe un programa Java con la clase `Main`. Alguien ha perdido las llaves y registra los bolsillos de su ropa, guardados en un array de `String`. Debes crear este método dentro de `Main`:

```
static void registrar(String[] bolsillos, String objetivo)
```

El método debe seguir estas reglas:

- Declara `int pos = -1`.
- Recorre el array con un `for` con índice. En cada posición imprime `Registrando: bolsillo`.

- Si el bolsillo es igual a `objetivo` (usa `equals`), guarda el índice en `pos` y sal del bucle con `break`.
- Después del bucle, si `pos` es 0 o mayor, imprime `¡Llaves encontradas en X (posición P)!`. `X` es el valor de `objetivo` y `P` es `pos`.
- Si no, imprime `Las llaves siguen perdidas tras registrar N bolsillos.`, donde `N` es la longitud del array.

En `main` crea el array `bolsillos = {"abrigo", "pantalón", "mochila", "zapato", "gorro"}`. El orden de impresión en `main` es:

1. Imprime `Registro 1:`.
2. Llama a `registrar(bolsillos, "mochila")`.
3. Imprime `Registro 2:`.
4. Llama a `registrar(bolsillos, "cartera")`.

Salida prevista:

text

```
Registro 1:
Registrando: abrigo
Registrando: pantalón
Registrando: mochila
¡Llaves encontradas en mochila (posición 2)!
Registro 2:
Registrando: abrigo
Registrando: pantalón
Registrando: mochila
Registrando: zapato
Registrando: gorro
Las llaves siguen perdidas tras registrar 5 bolsillos.
```

Primo 4: Menú de cumpleaños sin brócoli (`continue`)

Concepto: `continue` para saltar una vuelta y contadores.

Enunciado: escribe un programa Java con la clase `Main`. En un cumpleaños infantil se retira del menú todo el brócoli antes de servir. Debes crear este método dentro de `Main`:

```
static void servir(String[] menu)
```

El método debe seguir estas reglas:

- Declara dos contadores enteros, `servidos` y `escondidos`, ambos a 0.
- Recorre el array con un `for`.
- Si el plato es `"brócoli"` (usa `equals`), suma 1 a `escondidos` y pasa a la siguiente vuelta con `continue`.
- En los demás casos imprime `Servido: plato` y suma 1 a `servidos`.
- Después del bucle, si `servidos` vale 0, imprime `Mesa vacía: solo había brócoli.`
- Después imprime siempre `Brócolis escondidos: N`.

El orden de impresión en `main` es:

1. Imprime `Menú 1:`.
2. Llama a `servir` con el array `{"brócoli", "tarta", "brócoli", "refresco", "brócoli"}`.
3. Imprime `Menú 2:`.
4. Llama a `servir` con el array `{"brócoli", "brócoli"}`.

Salida prevista:

text

```
Menú 1:
Servido: tarta
Servido: refresco
Brócolis escondidos: 3
Menú 2:
Mesa vacía: solo había brócoli.
Brócolis escondidos: 2
```

PROMETEO
by thePower

Primo 5: Intérprete de gestos del mimo (`switch` con `return`)

Concepto: `switch` sobre `String`, `return` en cada `case` y `default`.

Enunciado: escribe un programa Java con la clase `Main`. Un mimo se comunica solo con gestos y necesitas traducirlos. Debes crear este método dentro de `Main`:

```
static String interpretar(String gesto)
```

El método debe usar un `switch` sobre el texto `gesto` con estas reglas. Cada `case` termina con `return`, sin `break`:

- `"ceja"` devuelve `Eso no me lo creo`.
- `"bostezo"` devuelve `Qué aburrimiento`.
- `"pulgar"` devuelve `Todo bajo control`.

- El texto vacío `""` devuelve `Cara de póker`.
- El caso `default` devuelve `Gesto desconocido`.

En `main` crea el array `gestos = {"ceja", "bostezo", "", "baile", "pulgar"}`. Recórrelo con un `for` y, para cada gesto, imprime una línea con el formato `Gesto 'X' = traducción`. Las comillas simples rodean al gesto, incluso cuando está vacío.

Salida prevista:

text

```
Gesto 'ceja' = Eso no me lo creo
Gesto 'bostezo' = Qué aburrimiento
Gesto '' = Cara de póker
Gesto 'baile' = Gesto desconocido
Gesto 'pulgar' = Todo bajo control
```

PROMETEO
by thePower